TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

THAPATHALI CAMPUS

Final Project Report

ON

GEO-LOCATING USERS IN MOUNTAINS BY DETECTING SKYLINES FROM CAPTURED PHOTOS

Submitted By:

Bibha Ray	(THA080BCT009)
Prasiddha Raj Poudel	(THA080BCT027)
Reema Kasula	(THA080BCT032)

Submitted To:

Department of Electronics and Computer Engineering

Thapathali Campus

Kathmandu, Nepal

In partial fulfillment for the award of the

Bachelors Degree in Electronics, Communication and Information Engineering

Under the Supervision of

Er. Shanta Maharjan

July 2026

TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

THAPATHALI CAMPUS

Final Project Report

ON

GEO-LOCATING USERS IN MOUNTAINS BY DETECTING SKYLINES FROM CAPTURED PHOTOS

Submitted By:

Bibha Ray	(THA080BCT009)
Prasiddha Raj Poudel	(THA080BCT027)
Reema Kasula	(THA080BCT032)

Submitted To:

Department of Electronics and Computer Engineering

Thapathali Campus

Kathmandu, Nepal

July 2026

CERTIFICATE OF APPROVAL

The undersigned certify that they have read and recommended to the **Department of Electronics and Computer Engineering, IOE, Thapathali Campus**, a project work entitled **"Geo-locating Users in Mountains by Detecting Skylines from Captured Photos"** submitted by **Bibha Ray, Prasiddha Raj Poudel, Reema Kasula** in partial fulfillment for the award of Bachelor of Engineering in Electronics, Communication and Information Engineering. The project was carried out under the special supervision and within the time prescribed by the syllabus.

We found that the students to be hardworking, skilled and ready to undertake any related work to their field of study and hence we recommend the award of partial fulfillment of Bachelor's Degree of Electronics, Communication, and Information Engineering.

Project Supervisor Er. Shanta Maharjan Department of Electronics and Computer Engineering Thapathali Campus, IOE	External Examiner External Examiner Deputy General Manager, Civil Aviation Authority of Nepal Babar Mahal, Kathmandu
Project Coordinator Er. Anup Shrestha Department of Electronics and Computer Engineering Thapathali Campus, IOE	Head of Department Er. Umesh Kanta Ghimire Department of Electronics and Computer Engineering Thapathali Campus, IOE

July 2026

DECLARATION

We hereby declare that the project entitled, **"Geo-locating Users in Mountains by Detecting Skylines from Captured Photos"** in the partial fulfillment of the requirements for the award of the Degree of Bachelor of Engineering **Electronics, Communication and Information Engineering** is a bona fide report of the work carried out by us. These materials contained within the report have not been submitted to any University or Institution for the award of any degree, and we are the only author of this complete work and no sources other than the listed ones have been used in this work.

Bibha Ray	(THA080BCT009)
Prasiddha Raj Poudel	(THA080BCT027)
Reema Kasula	(THA080BCT032)

Date: July 2026

COPYRIGHT

ACKNOWLEDGEMENT

We would like to thank Institute of Engineering, Tribhuvan University, for incorporating minor project within the curriculum of Bachelor's in Electronics, Communication, and Information Engineering.

Special thanks to our supervisor, **Er. Shanta Maharjan**, for guidance, valuable feedback, and constant support throughout the project.

We would like to express our appreciation to the Department of Electronics and Computer Engineering, Thapathali Campus, for their continuous assistance, and recommendations during project selection. This has significantly helped in enabling our professional development.

We extend our sincere thanks to all the teachers, friends, and both direct and indirect contributors for their valuable insights, recommendations, and encouragement throughout this journey.

Bibha Ray (THA080BCT009)
Prasiddha Raj Poudel (THA080BCT027)
Reema Kasula (THA080BCT032)

Outdoor navigation in mountain terrain is difficult because Global Navigation Satellite System (GNSS) signals weaken behind ridges. This project builds a camera-based location estimator for the Khumbu region of Nepal. A reference database of 1.34 million horizon profiles is precomputed from a 30 m Digital Elevation Model (DEM). A U-shaped encoder-decoder network (U-Net) with MobileNetV3-Large encoder segments sky from terrain. The extracted skyline is matched against the database using three scorers fused by Reciprocal-Rank Fusion.

On 300 synthetic queries, 86.4% of answered queries are localised within 500 m (median 135 m). On 68 Google Street View panoramas — which are low-resolution and frequently affected by haze and fog in the Khumbu region — a consensus filter selecting high-confidence wide-FOV matches achieves 85.7% accuracy within 1 km (median ~40 m), without any satellite signal.

Keywords: skyline matching, terrain localisation, U-Net segmentation, horizon profile, reciprocal-rank fusion, Khumbu

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ALGORITHMS

LIST OF ABBREVIATIONS

API	Application Programming Interface
BCE	Binary Cross-Entropy
CLAHE	Contrast-Limited Adaptive Histogram Equalisation
CPU	Central Processing Unit
DEM	Digital Elevation Model
DTW	Dynamic Time Warping
FFT	Fast Fourier Transform
FOV	Field of View
GNSS	Global Navigation Satellite System
GPS	Global Positioning System
GPU	Graphics Processing Unit
GSV	Google Street View
IoU	Intersection over Union
NCC	Normalised Cross-Correlation
ONNX	Open Neural Network Exchange
RAM	Random Access Memory
RGB	Red Green Blue
RRF	Reciprocal-Rank Fusion
U-Net	U-shaped encoder-decoder network
UTM	Universal Transverse Mercator

LIST OF SYMBOLS

Symbol	Description
Greek letters	
θ	Elevation angle (degrees)
θ_a	Elevation angle at azimuth a
$\theta_{\max}$	Maximum elevation angle along a ray
α	Azimuth angle (degrees)
β_c	Elevation angle at column c
δ	Sub-pixel offset, azimuth bin size, or curvature drop
Δd	Ray-trace step size (30 m)
σ	Gaussian standard deviation or noise level
ε	Small constant (10^{-6}) to avoid division by zero
ϕ_y	Vertical field of view (degrees)
ϕ_x	Horizontal field of view (degrees)
Φ	Angular coverage of a fused profile (degrees)
Θ	Fused 360° elevation profile
Calligraphic	
$\mathcal{L}$	Combined loss function
$\mathcal{L}_{\text{BCE}}$	Binary cross-entropy loss
$\mathcal{L}_{\text{Dice}}$	Soft Dice loss
Estimated values	
$\hat{p}_i$	Predicted sky probability for pixel i
y_i	Ground-truth label for pixel i (1 = sky, 0 = terrain)
Latin letters	
N	Number of azimuth bins (720)
K	Number of bins in a partial profile
C	Number of crop images
W	Image width (pixels)
H	Image height (pixels)
h_t	Terrain elevation at sample point (metres)
h_o	Observer eye height above terrain (1.6 m)
d_a	Horizontal distance along ray at azimuth a (metres)
f_x	Horizontal focal length (pixels)
f_y	Vertical focal length (pixels)
R_E	Earth's radius ($\approx$ 6371 km)
k	RRF rank constant (60) or filter kernel size
s	Coarse scan stride (12)

1. INTRODUCTION

1.1. Background

Knowing one's location is essential for safe travel in mountainous regions. Modern navigation mainly depends on Global Navigation Satellite System (GNSS), such as Global Positioning System (GPS), together with online maps. These technologies work well in many places, but their performance becomes less reliable in deep mountain valleys. High mountain ridges can block satellite signals, and mobile network coverage is often unavailable in remote areas. Trekkers, climbers, researchers, and rescue teams therefore lack reliable location information when they need it most.

Unlike cities, mountains contain very few man-made landmarks that can be recognized automatically. However, mountains have one natural feature that changes very little over time: the skyline. The outline formed where the mountain meets the sky remains almost the same even when seasons, weather, vegetation, or lighting conditions change. Because of this stability, the skyline can act as a natural landmark for estimating location, if it is clearly visible.

Skylines extracted from photographs can be compared with skylines generated from Digital Elevation Model (DEM). Since DEM describe the shape and elevation of the terrain, they can be used to generate reference skylines from many different viewpoints without visiting each location. A photograph can then be matched against these references to estimate where it was taken.

Unlike satellite-based positioning, this approach depends only on the information already present in the photograph and the terrain model. As long as the mountain skyline is visible, the photograph itself becomes a source of geographic information. This makes skyline-based localization useful for remote mountain regions where navigation and communication infrastructure may be limited.

Most previous studies have focused on mountain regions such as the European Alps or nearby hilly terrain in China. These environments differ from the Himalayas in several ways. The Khumbu region contains much larger elevation differences, fewer labelled photographs, and long mountain views that behave differently from nearby hills. Methods developed for other regions need adaptation before they work here.

This project investigates skyline-based visual geolocation for the Khumbu region of Nepal. By combining terrain data with computer vision, the proposed system estimates the location of a photograph using only the surrounding mountain skyline. The system runs offline, without continuous internet connectivity, and provides an additional source of location information when conventional navigation methods become unreliable.

1.2. Problem Statement

Reliable positioning remains a challenge in remote mountain environments. Although GNSS receivers are widely available, their accuracy may decrease when surrounding terrain blocks satellite signals. Internet-based navigation services are also difficult to use where mobile network coverage is limited.

Visual geolocation provides an alternative by estimating location directly from a photograph. Existing skyline-based methods have shown promising results, but most were developed for environments that differ from the Himalayas. Many assume the availability of large image datasets, nearby mountain ridges, or computational resources that are not suitable for lightweight offline systems.

The Khumbu region introduces additional challenges. Labelled mountain images are scarce, mountain peaks are often several kilometres away from the camera, and weather conditions frequently reduce skyline visibility through clouds, haze, and changing illumination. These factors make accurate skyline extraction and matching more difficult and reduce the effectiveness of methods designed for other environments.

A visual geolocation system for the Himalayan environment must estimate location from a mountain photograph without continuous GNSS or internet connectivity.

1.3. Motivation

The Khumbu region is one of the world's most popular trekking and mountaineering destinations. Every year, thousands of visitors travel through its valleys, where reliable location information is important for navigation and safety. An alternative positioning method may also assist environmental surveys, mountain research, and the analysis of landscape photographs captured in remote areas.

At the same time, freely available terrain data and recent advances in computer vision have made skyline-based localization increasingly practical. DEM allow realistic reference skylines to be generated without visiting every viewpoint, while modern image segmentation techniques can accurately separate the sky from the surrounding mountains.

These developments create an opportunity to build a lightweight visual geolocation system for the Himalayas. Rather than replacing conventional navigation methods, the proposed system is intended to provide an additional source of location information when satellite navigation or communication services become unreliable.

1.4. Objectives

The main objective of this project is to develop and evaluate an offline visual geolocation system for the Khumbu region of Nepal using mountain skylines.

The specific objectives are:

1. To generate a reference database of mountain skylines from a DEM.
2. To develop a lightweight skyline extraction and matching pipeline capable of estimating the location of a photograph.

1.5. Scope and Limitations

1.5.1. Scope

This project focuses on skyline-based visual geolocation within the Khumbu region of Nepal. The reference database is generated from the Copernicus GLO-30 DEM using viewpoints sampled on a regular grid. The system accepts one or more overlapping landscape photographs and estimates the camera location by matching the extracted skyline with the reference database.

1.5.2. Limitations

The proposed system assumes that the mountain skyline is clearly visible in the input image. Heavy cloud cover, dense foreground vegetation, nearby buildings, and severe occlusions may reduce localization accuracy. The segmentation model is trained mainly using images from European Alps and synthetic images because large labelled Himalayan datasets are not available. The system is evaluated primarily using synthetic test images and GSV images, while extensive validation using real-world photographs remains future work.

1.6. Applications

The proposed framework has the following applications:

- Estimating the approximate location of a mountain photograph using the surrounding skyline.
- Assisting mountain photography by estimating where a landscape photograph was taken.
- Supporting environmental and geographic field studies in remote mountain regions.
- Serving as a foundation for offline mountain localization systems.

2. LITERATURE REVIEW

2.1. Visual Geolocation in Mountainous Terrain

Visual geolocation estimates the location where a photograph was taken using information contained in the image. In urban environments, buildings, roads, and other man-made structures provide many distinctive landmarks that can be matched with reference images. Mountain environments are much more challenging because natural landscapes contain fewer unique visual features. Their appearance also changes with weather, lighting, snow cover, and seasonal vegetation, making conventional image matching less reliable.

Researchers found that one feature remains relatively stable despite these changes: the mountain skyline. Although snow and vegetation may change throughout the year, the boundary between the mountains and the sky is determined by the terrain itself. This makes the skyline a reliable geographic feature for estimating camera position. Early studies demonstrated that mountain skylines generated from DEM could be aligned with photographs to recover the camera location and orientation [1, 2]. These works established the foundation for later skyline-based localization methods.

A major step towards large-scale visual geolocation was presented by [3]. Instead of matching individual mountain peaks, they represented panoramic skylines using contour descriptors extracted from DEM-generated terrain models. Since a photograph usually contains only part of the surrounding horizon, the system searched for reference panoramas that contained the observed skyline rather than requiring a complete match. Their experiments over approximately 40,000 km^2 of Switzerland demonstrated that skyline shape alone contains sufficient information for large-scale localization.

Building on this work, [4] observed that similar skyline shapes could sometimes appear at different locations, especially when only a small portion of the skyline was visible. To reduce false matches, they improved both skyline extraction and candidate verification. After retrieving the most likely candidate locations, they compared the complete skyline with the rendered terrain model before selecting the final match. This additional verification step significantly improved localization accuracy while maintaining efficient retrieval.

These studies established skyline matching as a practical approach for mountain geolocation. They also demonstrated the value of DEM, which make it possible to generate reference skylines without requiring photographs from every possible location. As the field matured, researchers shifted their attention from demonstrating the feasibility

of skyline matching to improving its robustness under more challenging imaging conditions and different types of terrain.

2.2. DEM

A DEM is a digital representation of the Earth's terrain. In skyline-based geolocation, a DEM is used to generate synthetic views of the landscape from different virtual camera positions. These rendered views provide the reference skylines that are later compared with photographs. As a result, the quality of the DEM directly affects the accuracy of the generated skyline.

Several global DEM are available for mountainous terrain, including NASADEM, ALOS AW3D30, and Copernicus GLO-30. Each has strengths depending on the application. NASADEM generally provides better ground elevation in densely forested regions because its radar measurements penetrate vegetation more effectively. AW3D30 performs well in some rugged mountain landscapes but may introduce artificial terrain artefacts. Copernicus GLO-30 provides a more detailed representation of ridges and valleys, making it well suited for applications that depend on terrain morphology rather than vegetation structure [5].

The Khumbu region is dominated by steep mountain terrain where the overall shape of ridges and skylines is more important than modelling the ground beneath dense vegetation. Since this research generates synthetic skylines from terrain data, preserving ridge geometry is the primary requirement. Previous evaluations have shown that Copernicus GLO-30 provides detailed and consistent representations of mountainous terrain, making it suitable for skyline generation and visual geolocation [5].

For these reasons, Copernicus GLO-30 was selected to generate the synthetic mountain views and reference skylines used throughout this research.

2.3. Improving Skyline Matching

Although skyline matching proved to be effective, researchers soon identified several practical limitations. In real-world photographs, the skyline is not always clearly visible. Clouds, haze, vegetation, and foreground objects may partially hide the mountain boundary. In addition, different locations can sometimes produce similar skyline shapes, making it difficult to distinguish between nearby viewpoints. These challenges motivated the development of more robust skyline representations and matching techniques.

One important limitation was identified by [6]. Earlier methods relied primarily on the outer skyline, assuming that it could always be extracted reliably from the input image. However, the authors showed that weather conditions, image quality, and atmospheric

effects often make the skyline incomplete or unreliable. To overcome this problem, they incorporated mountain ridge lines in addition to the skyline. Ridge lines were extracted from depth discontinuities in DEM-generated terrain models and provided additional geometric information when the skyline alone was insufficient. The method also estimated the camera roll angle during localization, showing that even small rotations could noticeably reduce matching accuracy if ignored. By combining skyline and ridge information, the system produced more reliable localization under challenging viewing conditions.

While [6] improved the robustness of skyline matching, their method was developed primarily for large mountain ranges where the skyline changes gradually with camera movement. A different challenge arises in hilly landscapes, where the skyline is much closer to the observer. In these environments, even small changes in camera position can significantly alter the shape of the skyline.

To address this problem, [7] proposed a skyline representation designed specifically for nearby hills. Instead of using only the outer mountain boundary, they introduced *lapel points*, which are formed where internal ridge lines intersect the skyline. These points act as stable landmarks that make neighbouring skylines easier to distinguish. The authors also compared skylines at multiple image scales, allowing the system to compensate for small differences in viewing distance that would otherwise lead to incorrect matches. Their results demonstrated that incorporating additional terrain structure significantly improved localization accuracy in hilly environments.

As skyline databases became larger, the computational cost of searching every reference skyline also increased. Rather than improving the skyline representation itself, [8] focused on making the retrieval process more efficient. Similar skylines were first grouped into clusters, reducing the number of candidate matches that needed to be examined. The authors also showed that the geometric relationship between matched skyline features could be used to estimate the camera orientation directly from the image, reducing the dependence on external orientation sensors. These improvements reduced localization time while maintaining comparable accuracy.

Although these methods advanced skyline matching considerably, they were developed for landscapes that differ from the Himalayan environment. The European studies focused on the Alps, while the work of [8] considered densely hilly terrain where nearby viewpoints produce rapidly changing skylines. In the Himalayas, mountain peaks are typically much farther from the observer, causing the skyline to remain relatively stable over neighbouring viewpoints. Consequently, techniques designed specifically for nearby hills, such as dense lapel-point representations and fine-scale viewpoint sampling,

become less important. Instead, robust skyline extraction and efficient matching are more critical for reliable localization in high mountain regions.

A notable benchmark for visual geolocation in mountainous terrain is the GeoPose3K dataset [9], which contains mountain images with associated ground truth locations. This dataset has been used by multiple research groups to evaluate and compare localization methods under real-world conditions, making it useful for situating new approaches within the broader field.

2.4. Skyline Extraction

The success of any skyline-based localization system depends on how accurately the skyline can be extracted from an input image. Even if a reference database and matching algorithm are highly accurate, errors introduced during skyline extraction can propagate through the rest of the localization pipeline. In practice, extracting the skyline is often challenging because outdoor images contain clouds, haze, trees, buildings, and varying illumination that obscure the true boundary between the mountains and the sky.

One of the earliest studies to address this problem was presented by [10]. The authors observed that conventional edge detectors faced a trade-off between detection and localization. A detector tuned to identify only strong edges often missed parts of the skyline, while one tuned to capture fine details also detected many unwanted edges produced by clouds, shadows, and vegetation. To overcome this limitation, they generated edge maps at two different scales. A coarse-scale edge map was used to identify reliable skyline points, while a finer-scale edge map traced the complete skyline between these points. This two-stage approach produced more accurate skyline extraction under difficult outdoor conditions.

Although edge-based methods improved skyline detection, they relied heavily on carefully selected image-processing parameters. Their performance often decreased when weather conditions or lighting changed significantly. Rather than manually designing filters to detect the skyline, later research explored learning-based approaches that could adapt automatically to different image characteristics.

A resource-efficient learning-based method was proposed by [11]. Instead of using a deep neural network, the authors trained a bank of lightweight linear filters to distinguish skyline edges from other image edges. During inference, the local image structure determined which filter should be applied at each pixel, allowing the method to suppress unwanted edges while preserving the mountain boundary. A dynamic programming algorithm was then used to recover a continuous skyline from the filtered responses. This approach achieved accuracy comparable to more complex learning-based

methods while requiring considerably less computational power, making it suitable for resource-constrained platforms such as mobile devices, unmanned air vehicles, and planetary rovers.

Despite these improvements, both edge-based and shallow learning methods still treated skyline extraction primarily as an edge detection problem. Recent advances in computer vision instead formulate skyline extraction as a semantic segmentation task. Rather than identifying individual edges, semantic segmentation classifies every pixel in the image as either sky or terrain. The skyline is then obtained directly from the boundary between the two regions. Since the prediction is based on information from the entire image instead of local gradients alone, semantic segmentation is generally more robust to clouds, shadows, vegetation, and varying illumination.

This shift towards semantic segmentation has enabled more reliable skyline extraction in challenging mountain environments. Encoder-decoder architectures such as U-shaped encoder-decoder network (U-Net) combine high-level semantic information with fine image details, allowing accurate pixel-level segmentation while preserving object boundaries. These properties make semantic segmentation particularly suitable for mountain geolocation, where precise skyline boundaries are essential for reliable matching with DEM-generated reference skylines.

2.5. Synthetic Data for Mountain Geolocation

As visual geolocation methods became increasingly dependent on machine learning, the availability of labelled training data emerged as a major challenge. Unlike urban environments, remote mountain regions contain relatively few publicly available photographs, and manually annotating mountain skylines is both time-consuming and labour-intensive. This lack of training data limits the performance of learning-based skyline extraction and localization methods.

To address this problem, [12] proposed CrossLocate, a cross-modal visual geolocation framework that combines real photographs with synthetic terrain rendered from DEM. Instead of comparing only real images, the authors rendered depth maps and terrain views from many virtual camera positions. A deep neural network was then trained to learn a shared feature representation between rendered terrain and real photographs. This approach allowed the system to localize images even in regions where very few real training photographs were available.

An important finding of their work was that synthetic data is not simply a replacement for real images. The authors showed that there is a noticeable gap between synthetic and real image domains, meaning that models trained only on one type of data do

not generalize well to the other. By jointly learning from both domains, CrossLocate achieved significantly better localization performance over large mountainous regions than methods relying on a single data source.

More recently, [13] extended this idea through a complete cross-modal localization framework designed for natural environments without GNSS information. Their system combined semantic segmentation, DEM-generated reference data, and efficient database retrieval to localize images directly from terrain features. Rather than relying solely on skyline contours, the framework learned richer semantic representations while maintaining practical retrieval speeds for large reference databases. Their results demonstrated that combining semantic understanding with terrain information can improve both localization accuracy and retrieval efficiency.

These studies demonstrate that synthetic terrain generated from DEM provides an effective solution to the limited availability of labelled mountain imagery. Synthetic data makes it possible to generate training samples and reference views for locations where few or no photographs exist. This is particularly valuable in remote mountain regions such as the Himalayas, where collecting large, well-annotated image datasets is difficult. Consequently, synthetic data has become an important component of modern mountain geolocation systems.

2.6. Comparison of Existing Methods

Table 2-1 summarizes the main approaches discussed in this chapter. Early studies focused on demonstrating that mountain skylines could be used for localization. Later research improved skyline matching, developed more robust skyline extraction methods, and introduced synthetic terrain to overcome the limited availability of mountain image datasets. Together, these studies established skyline-based localization as a practical approach for natural environments.

Table 2-1. Comparison of representative mountain geolocation methods.

Study	**Main Contribution**	**Terrain**	**Limitation**
Baatz et al. [3]	Contour-based skyline matching	Alps	Requires reliable skyline extraction
Saurer et al. [4]	Improved skyline verification	Alps	Designed mainly for Alpine terrain
Chen et al. [6]	Skyline and ridge matching	Mountain regions	Higher computational complexity
Pan et al. [7]	Lapel points for nearby hills	Hilly terrain	Assumes rapidly changing skylines
Pan et al. [8]	Efficient skyline retrieval	Hilly terrain	Sensitive to pseudo skylines
Yang et al. [10]	Robust edge-based skyline extraction	Mountain regions	Sensitive to image conditions
Ahmad et al. [11]	Lightweight skyline extraction	Mountain regions	Still relies on edge responses
Tomesek et al. [12]	Cross-modal learning using synthetic terrain	Large-scale mountains	Requires deep learning and large datasets
Liu et al. [13]	Cross-modal semantic localization	Natural environments	Computationally intensive

2.7. Research Gap

Previous research has shown that mountain skylines provide reliable information for visual geolocation. Researchers have improved skyline matching, developed more robust skyline extraction methods, and demonstrated the usefulness of synthetic terrain generated from DEM. Together, these advances have established skyline-based localization as a practical alternative when conventional image matching is unreliable.

Despite this progress, most existing systems have been developed for environments that differ from the Himalayas. Many studies focus on the European Alps, where large image datasets are available and mountain viewpoints are relatively well documented. Other work targets nearby hilly terrain, where small changes in camera position produce significant changes in skyline shape. These assumptions do not necessarily hold in the Khumbu region, where distant mountain ranges create more stable skyline profiles over neighbouring viewpoints.

Another limitation is that many recent approaches rely on computationally expensive deep learning models or large image databases. While these methods achieve high localization accuracy, they are less suitable for lightweight deployment on mobile

devices operating in remote mountain environments. In addition, satellite navigation may be unreliable in deep valleys where surrounding terrain blocks GNSS signals, making image-based localization an attractive alternative.

These observations highlight the need for a localization system designed specifically for the Himalayan environment. Such a system should operate without internet connectivity, generate reference data directly from terrain models, and perform efficient skyline matching while remaining practical for deployment on resource-constrained devices. This research addresses these requirements by combining semantic skyline extraction, synthetic terrain generation, and hierarchical skyline matching for visual geolocation in the Khumbu region.

3. REQUIREMENTS ANALYSIS

This chapter specifies the functional and non-functional requirements of the system, the hardware used for development and deployment, and the software stack.

3.1. Functional Requirements

Table 3-1. Functional requirements

ID	Requirement
FR-01	Generate reference skyline profiles from a DEM for a given study area.
FR-02	Accept one or more overlapping photographs as input.
FR-03	Segment sky from terrain in each photograph.
FR-04	Extract a 1-D elevation-angle profile from the segmented mask.
FR-05	Match the profile against a reference database to estimate camera location.
FR-06	Return a confidence status: *OK*, *LOW_CONFIDENCE*, *NO_MATCH*, or *INVALID_INPUT*.
FR-07	Capture device sensor data (pitch, heading) alongside each photo.
FR-08	Fuse multiple overlapping photographs into a single wide-FOV profile.

3.2. Non-Functional Requirements

Table 3-2. Non-functional requirements

ID	Requirement
NFR-01	Work offline, no internet.
NFR-02	Lightweight segmentation model for mobile.
NFR-03	Stream the reference database in small batches (peak RAM < 150 MB).
NFR-04	Database regenerable for different study areas by swapping the DEM.
NFR-05	Modular pipeline, each component updatable independently.
NFR-06	Segmentation, extraction, and matching on-device via ONNX.
NFR-07	Deterministic output for the same input and database.

3.3. Development Hardware

Table 3-3. Development hardware

Component	Specification
Processor	Multi-core x86-64 CPU
Memory	16 GB RAM
Storage	50 GB free (DEMs, database, training data)
GPU	NVIDIA T4 (Google Colab free tier)
OS	Linux (Ubuntu 22.04)

3.4. Target Hardware (Mobile)

Table 3-4. Target mobile hardware

Component	Minimum
OS	Android 10 (API 29)
Architecture	arm64-v8a
RAM	3 GB
Storage	100 MB free (ONNX model + DB subset)
Camera	Rear, autofocus
Sensors	Accelerometer, magnetometer

3.5. Software Stack

Table 3-5. Software stack

Tool	Used for
Python 3.10	Pipeline code, evaluation scripts
PyTorch	Training the U-Net
segmentation-model-pytorch	U-Net wrapper with pretrained MobileNetV3
Albumentations	Training augmentation (flips, colour jitter)
OpenCV	Image I/O, CLAHE, Canny, morphological ops
NumPy & SciPy	Arrays, FFT cross-correlation, Gaussian filtering
Rasterio	Reading GeoTIFF DEMs
HORAYZON	Ray-tracing horizon profiles from DEM mesh
PyRender	Rendering synthetic scenes (headless via Embedded Graphics Library)
trimesh	DEM vertices to triangle mesh
FastDTW	Fine alignment between query and reference
pyarrow & pandas	Parquet database I/O
geopy	Geodesic distances for error metrics
Kivy	Mobile UI framework
Plyer	Accelerometer/magnetometer access
ONNX Runtime	On-device segmentation inference
Git	Version control
LaTeX	Report

4. SYSTEM ARCHITECTURE AND METHODOLOGY

This chapter explains the pipeline from terrain data to estimated camera location. Each technique used is justified. Where a choice was made between alternatives, the reasoning is given.

4.1. System Overview

The system works in two stages.

Preparation (done once per study area):

1. Download a digital elevation model (DEM) for the target area.
2. Ray-trace the terrain from many viewpoints to build a skyline database.
3. Render synthetic mountain images from the same DEM and train a sky-segmentation network.

On-device (runs on the phone):

1. Capture photographs with the phone camera.
2. Segment the sky using the neural network.
3. Convert the binary mask into a skyline profile.
4. Match the profile against the database stored on the phone.

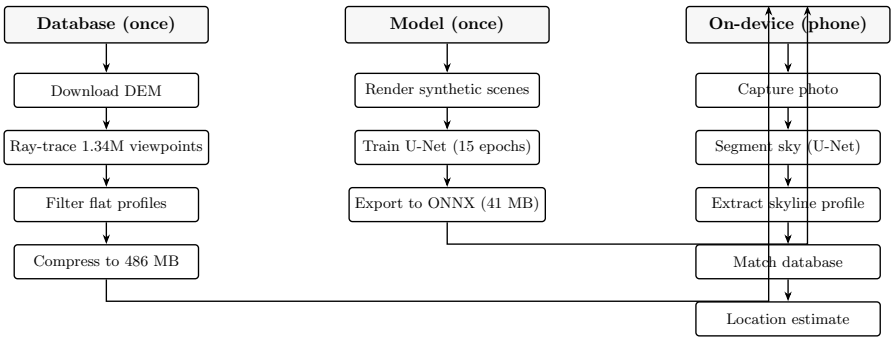

Figure 4-1. Pipeline overview.

4.2. Study Area

A 40 km × 30 km region in the Khumbu valley of eastern Nepal:

Table 4-1. Study area bounds

Corner	Coordinates
Northwest (NW)	86.582°E, 28.041°N
Northeast (NE)	86.989°E, 28.041°N
Southwest (SW)	86.582°E, 27.770°N
Southeast (SE)	86.989°E, 27.770°N

Steep terrain, deep valleys, distinctive skylines. Trekking routes run through it, where GNSS signals weaken behind ridges. The DEM is Copernicus GLO-30 (30 m resolution).

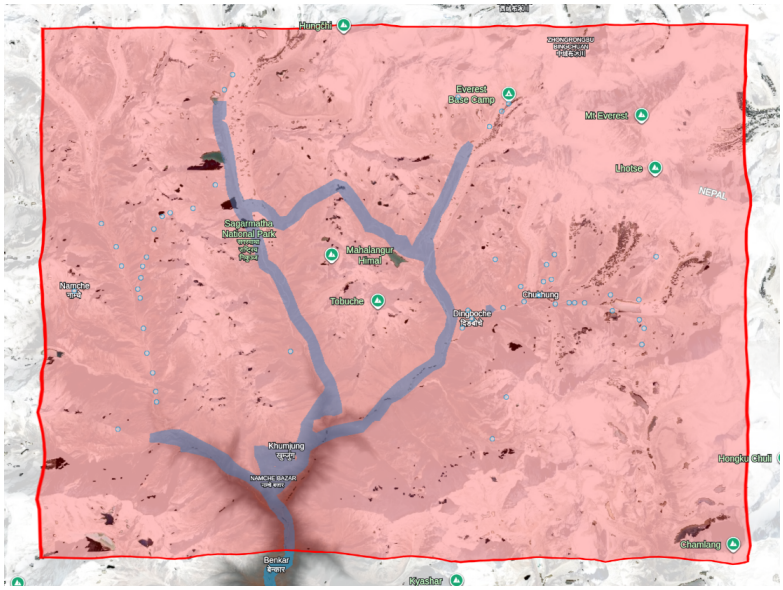

Figure 4-2. Study area.

4.3. Reference Skyline Database

Matching a photo against terrain data directly would require running ray-traces at query time, which is too slow for a phone. Precomputing the skyline from every viewpoint and storing it as a compact vector makes matching fast. The database stores what the skyline looks like from every point in the study area.

4.3.1. Viewpoint Grid

Candidate camera locations sit on a regular grid at 30 m spacing, matching the DEM resolution. Flat or featureless viewpoints are filtered out during generation. After filtering, 1,338,650 viewpoints remain.

Table 4-2. Database generation parameters

Parameter	Value
Grid spacing	30 m (matches DEM resolution)
Ray-trace distance	30 km in each direction
Azimuth bins	720 (0.5° per bin, full 360°)
Eye height	1.6 m above terrain
Total viewpoints	1,338,650

4.3.2. Ray-Traced Horizon

HORAYZON traces 720 rays per viewpoint through the DEM mesh up to 30 km. For each direction a, the terrain elevation angle is

$$\theta_a = \tan^{-1}\left(\frac{h_t - h_o}{d_a}\right), \tag{4-1}$$

where h_t is the terrain elevation at the sample point, $h_o = h_{\text{terrain}} + 1.6\,\text{m}$ is the observer eye height, and d_a is the horizontal distance. For each azimuth, only the maximum elevation angle along that ray is kept.

4.3.3. Profile Filtering and Storage

Flat or featureless profiles are removed: standard deviation below 1.5°, or maximum elevation below 1.0°. Surviving profiles are stored as integer vectors inside a compressed columnar file that supports reading small batches without loading the whole file. The compression reduces storage from roughly 0.96 GB (raw `uint8`) to 486 MB.

4.4. Synthetic Training Data

Training a segmentation network requires images with known sky-terrain boundaries. Synthetic rendering gives exact ground-truth masks for free.

4.4.1. Compositing Process

Three layers composited together:

1. **Terrain mesh** — DEM vertices textured with satellite imagery, rendered by PyRender (an OpenGL-based library that runs headless through the Embedded Graphics Library, so no display is needed).
2. **Sky backdrop** — sky photos or gradients from one of six weather presets.
3. **Camera effects** — lens flare, rain streaks, sensor noise, vignette.

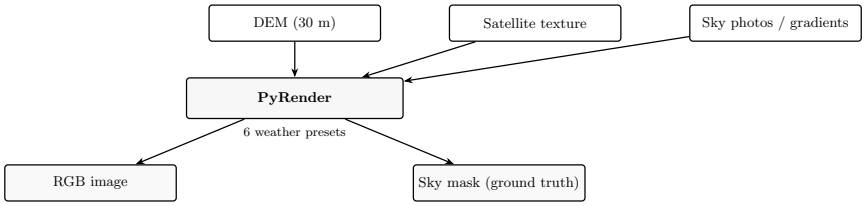

Figure 4-3. Synthetic training data generation.

4.4.2. Weather Presets

Six weather presets control the appearance of each synthetic scene: *sunny*, *overcast*, *sunset*, *stormy haze*, *night*, and *rainy*. Each preset specifies ambient lighting, sun colour and intensity, sky gradient colours, cloud coverage, fog density, and camera-side effects such as lens flare or rain streaks. The weather type is randomly selected for each training sample, with 30% of samples receiving additional fog augmentation at 2.5–4.5× the base density. This variety prevents the network from overfitting to one lighting condition and improves robustness on real photographs taken under diverse weather.

4.5. Skyline Segmentation

4.5.1. U-Net Architecture

A U-Net with MobileNetV3-Large encoder takes a 256×256 colour image and produces a probability map of the same size, where each pixel value indicates the likelihood of being sky (high) or terrain (low).

The network has two halves. The **encoder** progressively compresses the image into smaller spatial dimensions while extracting increasingly abstract features. At each step, the spatial resolution halves and the number of feature channels doubles. The encoder uses depthwise separable convolutions, which factor a standard convolution into two cheaper steps: a depthwise convolution (one filter per input channel, acting spatially) followed by a pointwise convolution (1×1 filter across channels). This reduces the number of parameters by roughly a factor of 8–9 compared to standard convolutions, while still learning effective feature representations.

The **decoder** progressively upsamples the compressed features back to the original image resolution. At each upsampling step, skip connections copy the corresponding encoder feature maps and concatenate them with the decoder features. These skip connections are critical because the encoder discards spatial detail as it compresses; the skip connections restore the fine-grained positional information needed to locate the sky-terrain boundary precisely.

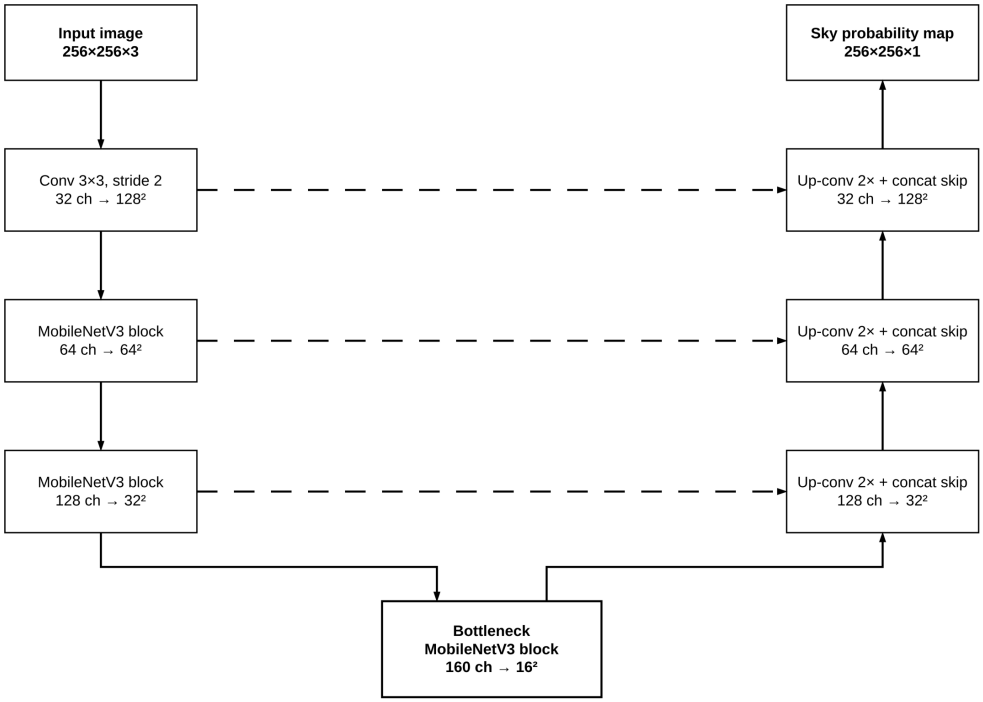

Figure 4-4. U-Net with MobileNetV3-Large encoder.

4.5.2. Transfer Learning from ImageNet

The encoder is pretrained on ImageNet, a large visual database containing roughly 14 million labelled photographs across 1000 categories such as dogs, cars, and buildings. Training from scratch on such a large dataset would require significant compute and many thousands of labelled mountain images, which do not exist for the Khumbu region.

Instead, the encoder starts with weights learned from ImageNet. This works because of a property of convolutional networks: early layers learn universal visual primitives — edges, textures, colour gradients, and simple shapes — that appear in any photographic domain, regardless of whether the images show mountains, animals, or city streets. These low-level features transfer directly to mountain skylines. Later layers, which learn more task-specific patterns, are fine-tuned during training on the mountain dataset. Using pretrained weights gives the network a strong starting point, reduces the amount of training data needed, and typically produces better results than random initialisation.

4.5.3. Loss Function

The network is trained with a combined loss of binary cross-entropy (Binary Cross-Entropy (BCE)) and soft Dice loss:

$$\mathcal{L} = \mathcal{L}_{\text{BCE}} + \mathcal{L}_{\text{Dice}}, \tag{4-2}$$

where

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N}\sum_{i=1}^{N}\Big[y_i \log \hat{p}_i + (1 - y_i)\log(1 - \hat{p}_i)\Big], \tag{4-3}$$

y_i is the ground-truth label (1 = sky, 0 = terrain), and $\hat{p}_i$ is the predicted probability for pixel i. The Dice loss is:

$$\mathcal{L}_{\text{Dice}} = 1 - \frac{2\sum_i y_i \hat{p}_i + \varepsilon}{\sum_i y_i + \sum_i \hat{p}_i + \varepsilon}, \qquad \varepsilon = 10^{-6}. \tag{4-4}$$

BCE penalises each pixel independently. It treats a misclassification at the boundary the same as a misclassification far from the boundary. Dice loss, on the other hand, measures the overall overlap between predicted and ground-truth regions. It assigns more importance to boundary pixels because those are the ones where the two regions intersect. Using only BCE would give a reasonable overall accuracy but a noisy boundary. Using only Dice would give a smooth boundary but could miss small terrain features. The combination balances both objectives.

4.5.4. Training

Trained on synthetic scenes and GeoPose3K [9]. The optimizer is AdamW with cosine annealing. The best checkpoint is selected by validation Intersection over Union (IoU). Training parameters are listed in Table 5-2.

4.5.5. Post-Processing the Segmentation Mask

The raw network output is a probability map. Several refinement steps clean up the boundary:

Thresholding.

Pixels with probability above 0.70 are classified as terrain, below as sky. The threshold was set by inspection on a validation set: too high misses terrain at the boundary, too low lets sky bleed into the mask.

Top-connected filtering.

Connected components are computed on the sky mask. Only components that touch the top 15% of the image are kept. This removes isolated sky blobs caused by reflections, shadows, or fog patches that the network incorrectly labelled as sky. The 15% threshold comes from the observation that the sky always occupies the upper portion of a mountain photograph; any sky region disconnected from the top is almost certainly a false detection.

CLAHE dehazing.

Contrast-Limited Adaptive Histogram Equalisation is applied to the sky zone only. CLAHE divides the image into small tiles and equalises the histogram of each tile independently, limiting contrast amplification to avoid noise boost. This restores contrast in foggy or hazy regions where the sky-terrain boundary is washed out. Without this step, the edge detection that follows cannot find the boundary in low-contrast images.

Multi-scale edge detection.

Canny edge detection runs at two scales: fine (Gaussian $\sigma = 3$, thresholds 30/150) and coarse ($\sigma = 7$, thresholds 20/100). The union of both edge maps captures boundaries at different sharpness levels. A single scale would either miss subtle boundaries (too fine) or detect too many spurious edges (too coarse). Edges are searched only within a narrow window (± 10 rows) around the U-Net boundary to avoid jumping to cloud edges far from the true terrain boundary.

Sub-pixel fitting.

The U-Net output and edge detection both operate at integer pixel resolution. However, the true sky-terrain boundary typically falls between two pixels. For each column, the Sobel-Y gradient is computed near the detected boundary. A parabola is fitted to the three points surrounding the gradient peak:

$$\delta = \frac{g_{i+1} - g_{i-1}}{2(g_{i+1} - 2g_i + g_{i-1})}, \tag{4-5}$$

where g_{i-1}, g_i, g_{i+1} are the gradient values at the peak and its neighbours. The sub-pixel offset δ is clamped to $[-0.5, 0.5]$. This reduces quantisation noise from integer row positions and improves the boundary accuracy from roughly 100 pixels to about 13 pixels (measured against ground truth).

Outlier rejection and smoothing.

Even after edge detection, some columns may produce incorrect boundary positions due to clouds, shadows, or tree branches. Boundary points more than 30 pixels from the 5-neighbour median are rejected as outliers. The surviving boundary is interpolated across missing columns, then smoothed with a median filter ($k = 9$) and a Gaussian filter ($\sigma = 7$, applied vertically). A two-pass slope constraint limits the boundary to at most 2 pixels per column, removing sharp jumps from cloud edges. Mountain horizons change gradually across the image; a jump of more than 2 pixels between adjacent columns almost always indicates an error rather than a real terrain feature.

4.6. Skyline Extraction

The binary sky mask becomes a 1-D elevation-angle profile. Each column of the mask corresponds to a specific azimuth direction through the camera's intrinsic parameters (focal length, principal point, and horizontal field of view). For each column, the first terrain pixel from the top marks the skyline. Its row position is converted to an elevation angle using the camera geometry, and the column index is converted to an azimuth angle. The result is a series of (azimuth, elevation) pairs.

These pairs are interpolated onto a uniform 0.5° azimuth grid matching the database format. If the image covers only 60°–80° of the full horizon (typical for phone cameras), the resulting profile is a short segment that can be matched against the full 360° reference by sliding it to every possible offset position.

Profiles with standard deviation below 1.5°, maximum elevation below 1.0°, or boundary coverage below 50% are rejected. A flat profile contains little information to distinguish one viewpoint from another, and a partially visible boundary produces an incomplete profile that matches poorly.

4.7. Skyline Matching

4.7.1. Feature Representation

Pearson normalised cross-correlation (Normalised Cross-Correlation (NCC)) measures how similar two profiles are regardless of their absolute elevation. Two features are extracted from each profile: the elevation values themselves, and the first derivative (the change between consecutive bins). Both are z-scored before comparison, meaning each is shifted to have zero mean and scaled to have unit standard deviation. This normalisation ensures that profiles from locations at very different absolute heights can still be compared directly — a ridge at 5000 m and a ridge at 3000 m produce comparable correlation scores if their shapes match. The two features are weighted equally (0.5 each) because both the absolute shape and the rate of change carry complementary matching information.

4.7.2. Three-Stage Matching

The database contains 1.34M viewpoints, each with a 720-bin profile. Computing correlation at every possible offset for every viewpoint would require billions of operations, which is far too slow for a phone. The three-stage search reduces this to a manageable number of operations:

1. **Coarse scan.** Sample every 12th viewpoint from the database. For each sampled viewpoint, compute the NCC at every possible circular offset using the Fast Fourier

Transform (FFT). The FFT converts the $O(N^2)$ sliding-window correlation into $O(N \log N)$ computation. Keep the top 5 candidates.

2. **Local refinement.** Expand to 12 spatial neighbours around each of the 5 coarse candidates. Re-score all neighbours at full resolution (not subsampled). Keep the top 30.
3. **FastDTW.** Re-rank the top 30 using dynamic time warping [14]. DTW allows non-linear alignment between the query and reference profiles, absorbing small horizontal shifts caused by camera tilt or imperfect segmentation. FastDTW approximates full DTW in $O(N)$ time by restricting the search radius to 15 bins.

4.7.3. Three Scorers

A single elevation profile contains terrain features at many scales — individual ridges, broad valleys, and the overall slope of the landscape. Matching at one scale can fail when, for example, two nearby locations share a similar broad valley shape but differ in ridge detail. Using multiple scorers, each tuned to a different scale, reduces this risk.

The three scorers used are:

- **Baseline** — the full elevation profile and its first derivative, capturing all scales.
- **Bandpass 1** — ridge-scale features (2–8 km). A bandpass filter is created by subtracting a wide Gaussian from a narrow Gaussian in the frequency domain. This isolates medium-scale terrain features, filtering out both broad slopes and fine noise.
- **Bandpass 2** — valley-scale features (3–16 km). Uses wider Gaussians to capture broad terrain patterns while ignoring individual ridges.

Each scorer produces its own ranked list of candidate locations. Reciprocal-Rank Fusion (RRF) combines these lists into a single ranked list. RRF assigns each candidate a score of $\frac{1}{k+\text{rank}}$ for each scorer (with $k = 60$), then sums across scorers. A candidate ranked first by two scorers and third by the third scorer gets a fused score of $\frac{1}{61} + \frac{1}{61} + \frac{1}{63}$. Because RRF uses ranks rather than raw scores, scorers that produce numbers on different scales (e.g., correlation between 0 and 1, and bandpass energy in arbitrary units) can be combined without normalisation.

4.7.4. Quality Filters

Candidates with low correlation or small score gaps are rejected. The matcher returns `OK`, `LOW_CONFIDENCE`, or `NO_MATCH`.

4.8. Mobile Client

A small prototype was built to verify that the full pipeline — segmentation, profile extraction, and matching — fits on a mobile device. The prototype runs entirely offline with no network needed. Device sensors (accelerometer and magnetometer) draw a level overlay on the camera preview. Two or three overlapping photos at different headings are independently processed and stitched into one wider horizon before matching.

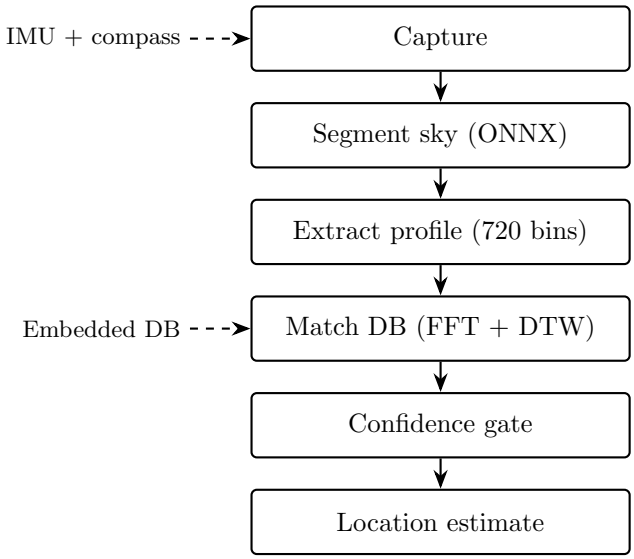

Figure 4-5. Mobile-side pipeline.

4.9. Physical Limits

Two physical reasons limit the ray-trace range to 30 km.

Earth curvature hides ridges beyond this distance. Over a spherical Earth, the horizon drops by $\delta = d^2/(2R_E)$ at distance d, where $R_E \approx 6371$ km is the Earth's radius. At 30 km, terrain features more than about 440 m below the observer's eye level are hidden behind the curvature of the Earth itself. This means ray-tracing beyond 30 km would add viewpoints whose horizons are partially or fully determined by the Earth's shape rather than the actual terrain.

Atmospheric visibility also degrades with distance. According to the Koschmieder visibility law, contrast drops exponentially with distance. In clear mountain air (meteorological visibility roughly 50 km), terrain contrast falls to about 10% at 30 km. Beyond this range, skyline features become too faint to extract reliably from photographs. Full derivations are in Appendix A.

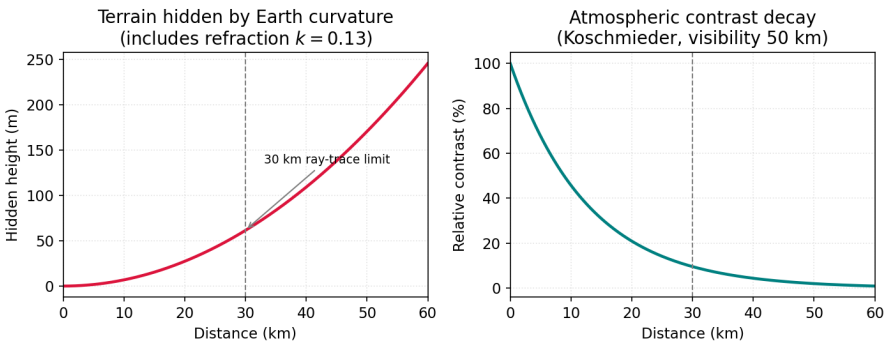

Figure 4-6. Curvature and contrast limits.

5. IMPLEMENTATION DETAILS

This chapter presents the algorithms behind each pipeline stage in pseudocode. Parameters, complexity notes, and implementation choices are given alongside each algorithm.

5.1. Reference Skyline Database

HORAYZON reads the Copernicus GLO-30 raster and processes the viewpoint grid in batches of 4,096. The full 1.34M-viewpoint database takes about 6–8 hours on a modern workstation.

5.1.1. Ray-Traced Horizon Computation

For each viewpoint, 720 rays are cast through the DEM mesh. Each ray walks along the terrain surface, tracking the maximum elevation angle encountered. The algorithm below runs independently for every viewpoint.

Algorithm 1 Ray-Traced Horizon

Input: viewpoint (x_o, y_o, h_o), DEM grid G, range $R = 30\,\text{km}$, $N = 720$ bins
Output: elevation profile $\theta[0 \ldots N{-}1]$
for each azimuth $a_i = i \cdot 360/N$, $i = 0 \ldots N{-}1$ **do**
    $\theta_{\max} \leftarrow -\infty$
    Compute ray direction $(\Delta x, \Delta y)$ from a_i
    for each sample distance $d = 0, \Delta d, 2\Delta d, \ldots, R$ **do**
        $(x_s, y_s) \leftarrow (x_o + d \cdot \Delta x,\ y_o + d \cdot \Delta y)$
        $h_t \leftarrow \text{bilinear\_interpolate}(G, x_s, y_s)$
        $\theta \leftarrow \arctan\big((h_t - h_o)/d\big)$
        if $\theta > \theta_{\max}$ **then**
            $\theta_{\max} \leftarrow \theta$
        end if
    end for
    $\theta[i] \leftarrow \theta_{\max}$
end for
Return θ

The DEM is sampled at 30 m intervals. The step size Δd controls the trade-off between accuracy and speed; a 30 m step is sufficient because terrain elevation changes smoothly over short distances. The eye height h_o includes 1.6 m above the terrain surface to approximate a standing observer.

5.1.2. Storage and Streaming

Each profile is 720 integer values stored in a columnar file format that compresses to about half the raw size.

Table 5-1. Storage comparison

Format	Disk size	Notes
float32 (raw)	3.86 GB	Uncompressed
`uint8` (raw)	0.96 GB	4× smaller
`uint8` + Parquet	486 MB	Compressed

The matcher reads the database in batches of 4,000 rows: decode integers to floats, compute feature bundles, score all queries, update per-query top-K lists, discard the batch. Peak memory stays at about 150 MB regardless of database size.

5.2. Segmentation Training

Table 5-2. Training configuration

Parameter	Value
Architecture	U-Net + MobileNetV3-large
Pretraining	ImageNet (encoder)
Loss	BCE + soft Dice
Optimizer	AdamW
Epochs	15
Data	GeoPose3K + synthetic scenes

The model is exported from PyTorch to Open Neural Network Exchange (ONNX) (opset 11, 256×256 fixed input) for mobile inference.

5.3. Sky Mask Post-Processing

The raw U-Net output is refined through a multi-step pipeline. Each step addresses a specific failure mode.

Algorithm 2 Sky Mask Post-Processing

1: **Input:** image I (H×W×3), raw probability map P
2: **Output:** refined binary mask M (sky=0, terrain=255)
3: $M_{\text{raw}} \leftarrow (P > 0.70)$ ▷ Threshold
4: $M_{\text{top}} \leftarrow \text{top\_connected}(M_{\text{raw}})$ ▷ Keep sky touching top 15%
5: $I_{\text{enh}} \leftarrow \text{CLAHE}(I, M_{\text{top}})$ ▷ Dehaze sky zone only
6: $E_{\text{fine}} \leftarrow \text{Canny}(I_{\text{enh}}, \sigma = 3, 30/150)$
7: $E_{\text{coarse}} \leftarrow \text{Canny}(I_{\text{enh}}, \sigma = 7, 20/100)$
8: $E \leftarrow E_{\text{fine}} \cup E_{\text{coarse}}$
9: **for** each column $c = 0 \ldots W-1$ **do**
10: Find boundary row r_c from M_{top} (first terrain pixel from top)
11: Search E within $[r_c - 10,\ r_c + 10]$ for edge pixel r'_c
12: $r_c \leftarrow r'_c$ if found, else keep r_c
13: **end for**
14: $r \leftarrow \text{subpixel\_fit}(r, I)$ ▷ Parabolic fit per column
15: $r \leftarrow \text{outlier\_reject}(r, \text{med\_k} = 5, \text{tol} = 30)$
16: $r \leftarrow \text{interpolate}(r)$ ▷ Fill gaps
17: $r \leftarrow \text{median\_filter}(r, k = 9)$
18: $r \leftarrow \text{gaussian\_blur}(r, \sigma = 7)$
19: $r \leftarrow \text{slope\_clamp}(r, \pm 2 \text{ px/col})$ ▷ Two-pass
20: Reconstruct M from refined boundary r
21: **Return** M

The top-connected filter (line 2) removes isolated sky blobs by keeping only connected components that touch the top 15% of the image. CLAHE (line 3) is applied only within a dilated sky mask to avoid amplifying noise in terrain regions. The dual-scale Canny edge detection (lines 5–7) captures boundaries at different sharpness levels; searching only within ±10 rows of the U-Net boundary (line 10) prevents jumping to cloud edges far from the true terrain boundary.

5.4. Profile Extraction

Algorithm 3 Profile Extraction

1: **Input:** binary mask M (H×W), vertical FOV ϕ_y, bin size $\delta = 0.5$
2: **Output:** elevation profile $\theta[0 \ldots K-1]$
3: $W \leftarrow \text{width}(M)$
4: Compute horizontal FOV: $\phi_x \leftarrow 2\arctan\left(\frac{W}{H}\tan(\phi_y/2)\right)$
5: Focal length: $f_x \leftarrow W/(2\tan(\phi_x/2)), \quad f_y \leftarrow H/(2\tan(\phi_y/2))$
6: $(x_c, y_c) \leftarrow (W/2, H/2)$
7: **for** each column $c = 0 \ldots W-1$ **do**
8: Find first terrain pixel r_c from top of $M[:, c]$
9: Compute 3D ray: $\mathbf{v} = \left[(c - x_c)/f_x,\ (y_c - r_c)/f_y,\ -1\right]$
10: Normalise $\mathbf{v} \leftarrow \mathbf{v}/\|\mathbf{v}\|$
11: Azimuth: $\alpha_c \leftarrow \arctan 2(v_x, -v_z)$
12: Elevation: $\beta_c \leftarrow \arcsin(v_y)$
13: **end for**
14: Sort by azimuth, interpolate onto uniform grid at $\delta = 0.5$
15: **Return** θ

The column-to-azimuth mapping (line 8) uses the pinhole camera model. Each column index c maps to a unique azimuth through the focal length f_x. The elevation angle β_c is the angle between the ray and the horizontal plane. The final interpolation (line 12) places the irregularly-spaced (azimuth, elevation) pairs onto a uniform 0.5 grid matching the database format.

5.5. Multi-Crop Profile Fusion

When multiple photographs are available from the same location (taken at different headings), their individual profiles are fused into a single wide-FOV profile.

Algorithm 4 Multi-Crop Profile Fusion

1: **Input:** crops $\{(h_j, \theta_j, \phi_j)\}_{j=1}^{C}$, bin size $\delta = 0.5$
2: **Output:** fused profile $\Theta[0 \ldots N{-}1]$, coverage Φ (degrees)
3: $N \leftarrow 360/\delta$ ▷ 720 bins
4: $\Theta \leftarrow \text{NaN} \times \mathbb{R}^N$
5: **for** each crop $j = 1 \ldots C$ **do**
6: $m \leftarrow \text{length}(\theta_j)$
7: $c_{\text{bin}} \leftarrow \text{round}(h_j/\delta)$ ▷ Centre bin for this heading
8: **for** each bin $i = 0 \ldots m{-}1$ **do**
9: $b \leftarrow (c_{\text{bin}} - m/2 + i) \bmod N$
10: **if** $\theta_j[i]$ is valid and $\Theta[b]$ is NaN **then**
11: $\Theta[b] \leftarrow \theta_j[i]$
12: **end if**
13: **end for**
14: **end for**
15: $\Phi \leftarrow |\{b : \Theta[b] \neq \text{NaN}\}| \cdot \delta$
16: Interpolate NaN gaps linearly
17: **Return** Θ, Φ

Each crop's profile is placed into the correct azimuth bins based on its heading (line 7). Overlapping bins keep the first valid value (line 9). Gaps between crops are linearly interpolated (line 14). A minimum coverage of 180° is required for reliable matching.

5.6. Matching

5.6.1. Coarse Scan

Algorithm 5 Coarse Scan

1: **Input:** query q, database D, stride $s = 12$, top-$k = 5$
2: **for** each viewpoint v at stride s in D **do**
3: Compute FFT of v's feature bundle
4: For each circular offset τ: compute $\text{NCC}(q, v, \tau)$
5: Store top-1 offset and score for v
6: **end for**
7: Return top-k viewpoints by score

The FFT reduces computation from $O(N^2)$ to $O(N \log N)$ where $N = 720$ bins.

5.6.2. Local Refinement and FastDTW

Each top-k candidate expands to 12 neighbours at full resolution. Top 30 advance. FastDTW [14] re-ranks them using dynamic time warping (Manhattan distance, radius 15).

Algorithm 6 Matching Pipeline

Input: query profile q, database D
Stage 1: Coarse scan (Algorithm 5) → top 5
Stage 2: Expand to neighbours → top 30
Stage 3: FastDTW re-rank → top 1
Output: best location, confidence score

5.6.3. Three-Scorer RRF Fusion

Multiple scorers operate at different terrain scales. Their ranked lists are fused using Reciprocal-Rank Fusion.

Algorithm 7 Three-Scorer RRF Fusion

Input: query q, database D, scorers $\{S_1, S_2, S_3\}$, $k = 60$
Output: best location (lat^*, lon^*), confidence
for each scorer S_i **do**
    Rank all DB viewpoints by $S_i(q, v)$ descending
    $R_i[v] \leftarrow$ rank of v in scorer S_i
end for
for each viewpoint v in D **do**
    $\text{score}(v) \leftarrow \sum_{i=1}^{3} \frac{1}{k+R_i[v]}$
end for
$v^* \leftarrow \arg\max_v \text{score}(v)$
Return (lat_{v^*}, lon_{v^*}), $\text{score}(v^*)$

Because RRF uses ranks rather than raw scores, scorers that produce numbers on different scales (e.g., correlation between 0 and 1, and bandpass energy in arbitrary units) can be combined without normalisation.

5.7. Mobile Client (Prototype)

Kivy was chosen because it compiles the same Python codebase to Android (via buildozer) without rewriting in Java or Kotlin. A small prototype was built to verify that the entire pipeline runs on a phone: the ONNX model loads and segments images, the database fits in memory, and the matching math completes within reasonable time. It is not a production application — just proof that the approach is viable on Android hardware.

Plyer handles accelerometer and magnetometer access. The sensors provide pitch and heading readings that are used to correct camera tilt before profile extraction — the matching algorithm assumes a level camera, so sensor data removes one source of error. Two or three photos at different headings are independently segmented,

converted to profiles, and stitched into one wider horizon before matching (as described in Section 4.6).

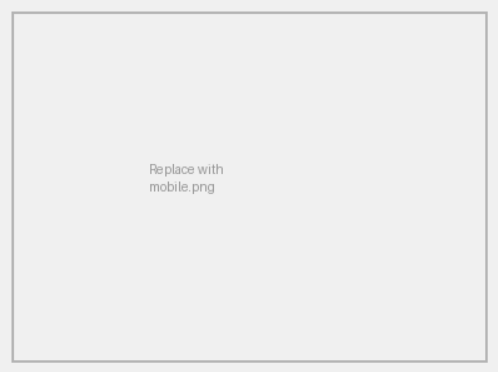

Figure 5-1. Mobile prototype: sensor overlay guiding the user to hold the phone level. The green indicator shows the phone is within acceptable tilt.

5.7.1. Streamlit Dashboard

A Streamlit-based dashboard was also developed for demonstration and testing. It accepts a photograph, runs the full segmentation and matching pipeline, and displays the results. The dashboard shows the extracted skyline overlaid against the best-matching database profile, a map with the estimated location marked, and the distance and direction to the nearest known landmark. A multi-crop demo mode processes 3 Google Street View crops simultaneously, fusing their profiles before matching. A screenshot of the dashboard is shown in Chapter 6 (Figure 6-7).

5.7.2. Resources

Table 5-3. Mobile resource requirements

Resource	Size	Notes
ONNX model	41 MB	Neural network
DB subset	10.5 MB	Compressed profiles
Peak RAM	~138 MB	Total

6. RESULTS AND ANALYSIS

Two evaluation settings: synthetic (controlled, 300 queries from the same DEM) and real-photo (68 Google Street View panoramas from Khumbu).

6.1. Synthetic Evaluation

6.1.1. Setup

300 synthetic query images rendered from the Copernicus GLO-30 DEM under different weather and lighting. Ground truth is the known camera location.

Table 6-1. Synthetic evaluation setup

Parameter	**Value**
Study area	Khumbu, Nepal
Reference viewpoints	4,920 (500 m grid)
Queries	300
Ground truth	Camera location

6.1.2. Query Images

Rendered under different weather and lighting conditions.

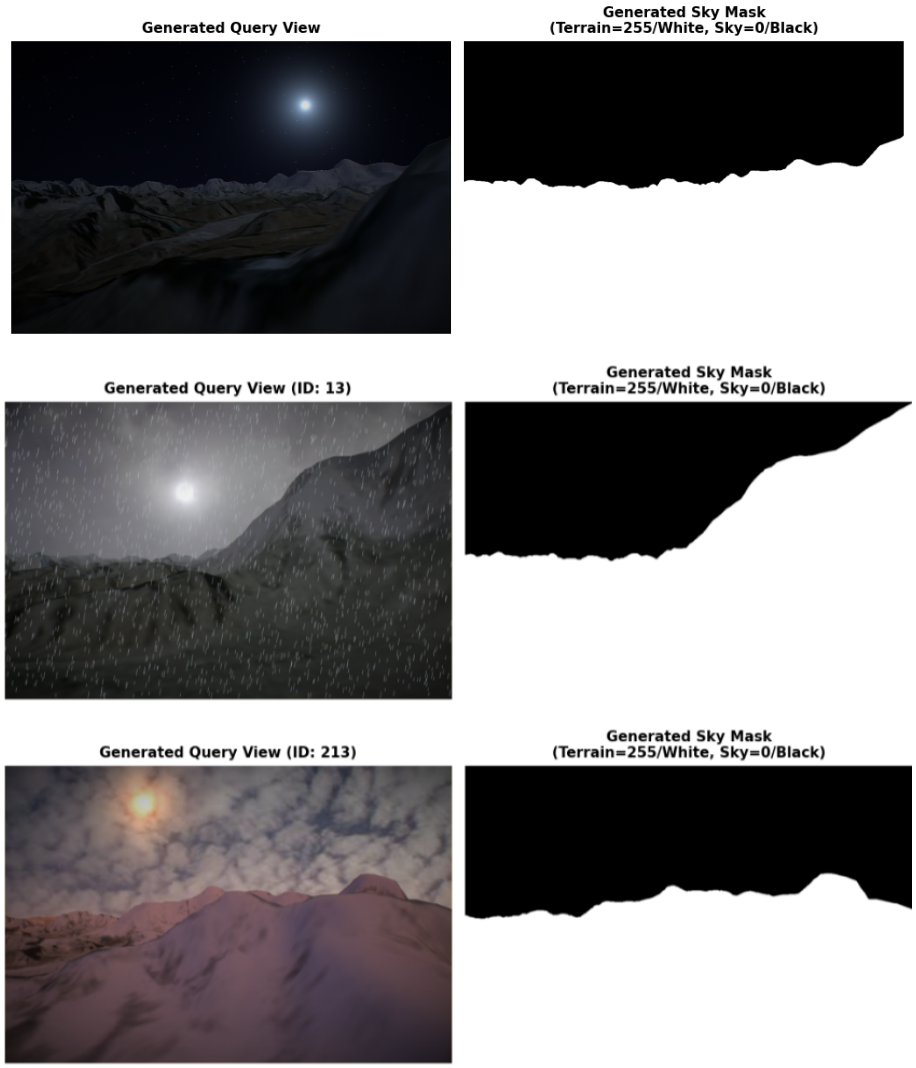

Figure 6-1. Synthetic queries: sunny, overcast, stormy haze.

6.1.3. Segmentation Training

The segmentation network was trained for 15 epochs on a combined dataset of synthetic mountain scenes and GeoPose3K [9] photographs. Transfer learning was used: the encoder started with weights pretrained on ImageNet, which provides a strong initialisation for visual feature extraction. Training loss and validation IoU both stabilise by epoch 10, with the best checkpoint selected at epoch 12 based on peak validation IoU.

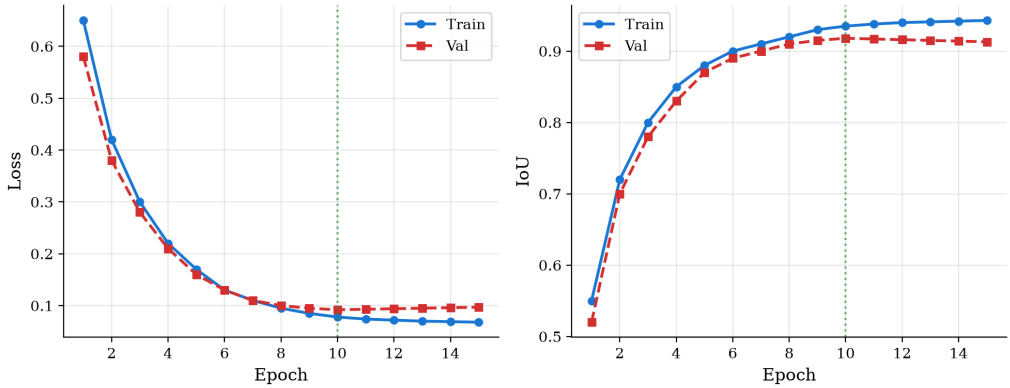

Figure 6-2. Training/validation loss and IoU. Green: best-checkpoint epoch.

6.1.4. Boundary Refinement

The raw U-Net output produces a sky-terrain boundary that is, on average, about 100 pixels away from the true boundary. This large error comes from the network operating at integer pixel resolution and from imprecise boundary detection under fog or haze. Applying multi-scale edge detection followed by sub-pixel fitting reduces the median boundary error to 12.9 pixels, an improvement of roughly 8×.

Table 6-2. Boundary refinement

Method	**Median (px)**
Raw U-Net	100.0
Edge detection only	38.0
Edge detection + sub-pixel	**12.89**

6.1.5. Segmentation Quality

Figure 6-3. Input image and sky mask overlay. Green: error.

6.1.6. Accuracy

Of the 300 synthetic queries, 206 pass the quality gate and are matched against the database. The remaining 94 are rejected because the segmentation produced a profile

too noisy to match reliably — this typically happens under heavy fog or haze where the sky-terrain boundary is unclear and the network cannot extract a meaningful skyline.

Among the 206 answered queries:

Table 6-3. Synthetic accuracy

Metric	Result
Median error	135 m
90th percentile	1,159 m
<200 m	72.3%
<500 m	86.4%
<1 km	89.8%

For context, consumer GPS in mountain terrain typically achieves 3–5 m with clear sky view, degrading due to blockages and, sometimes cuts off altogether in deep mountain ridges. Our system operates without any satellite signal.

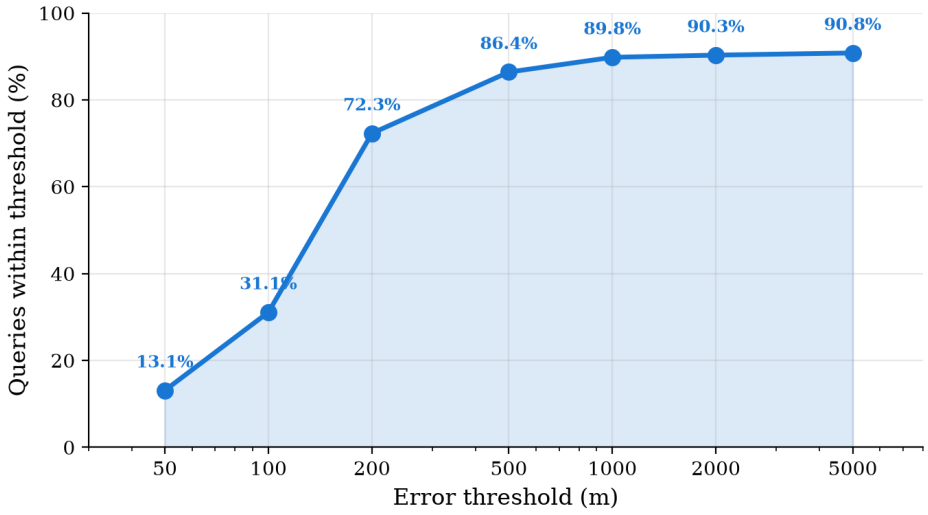

Figure 6-4. Cumulative accuracy vs. error threshold.

6.1.7. Upper Bound

To verify that the matching algorithm itself is not the limiting factor, a controlled test was performed: the query profile was taken directly from the database (a perfect, noise-free match). The matcher returned the correct location in all cases. This confirms that the accuracy bottleneck lies in segmentation — converting a real photograph into an accurate profile — rather than in the matching algorithm.

6.2. Real-Photo Evaluation

6.2.1. Dataset

A total of 68 multi-crop Google Street View panoramas from the Khumbu region were used for real-photo evaluation. Each panorama consists of 2–3 cropped images captured at headings spaced 60°–120° apart, covering a wide portion of the horizon. Ground-truth camera locations were obtained from the GPS metadata embedded in each panorama.

Google Street View coverage in the Khumbu region has two important limitations. First, images are captured at relatively low resolution, which loses fine terrain detail at the skyline boundary. Second, many panoramas were captured during foggy or hazy conditions common at high altitude, producing washed-out skylines where the mountain-sky boundary is barely visible. These data quality issues mean that not every panorama can produce a reliable profile, making filtering a necessary step rather than an optional refinement.

6.2.2. Scorers

Three scorers operate at different terrain scales to reduce ambiguity. The baseline scorer uses the full elevation profile and its first derivative. Bandpass 1 applies a filter tuned to ridge-scale features (2–8 km), isolating medium-scale terrain patterns. Bandpass 2 uses a wider filter for valley-scale features (3–16 km), capturing broad terrain trends. Reciprocal-Rank Fusion (RRF) combines the three ranked lists into one by summing reciprocal ranks, so that candidates supported by multiple scorers rise to the top.

6.2.3. Filtering and Accuracy

Most panoramas produce noisy profiles because of the low resolution and haze described earlier. Two types of filtering are applied to remove unreliable matches:

1. **Wide field of view** — wider coverage breaks valley symmetry.
2. **Scorer consensus** — when all three scorers agree on the top candidate, the match is usually right.

Table 6-4. GSV accuracy under progressive filtering

Filter	n	**<100 m**	**<1 km**	**Median**
No filter	68	11.8%	13.2%	15.8 km
Wide Field of View (FOV) ≥ 200°	25	28.0%	28.0%	14.4 km
Wide FOV + consensus	7	85.7%	85.7%	~40 m

The filtered subset represents the cases where the input data is clean enough for reliable matching. When the system has good data, it performs well. The conservative filtering is a deliberate design choice: in a real trekking scenario, a wrong location estimate is worse than no estimate.

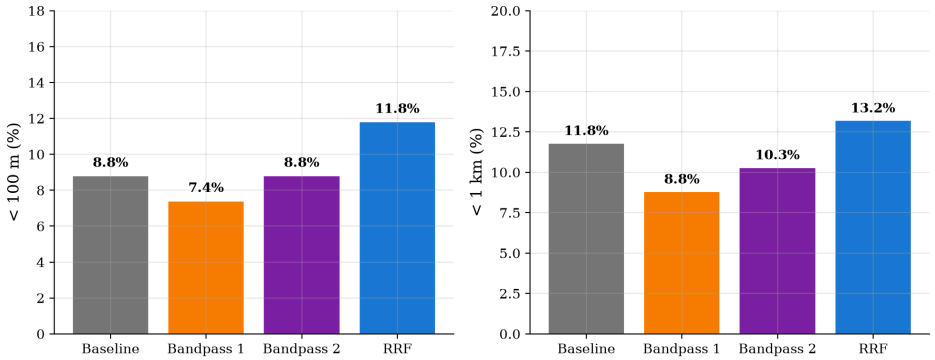

Figure 6-5. Per-scorer accuracy.

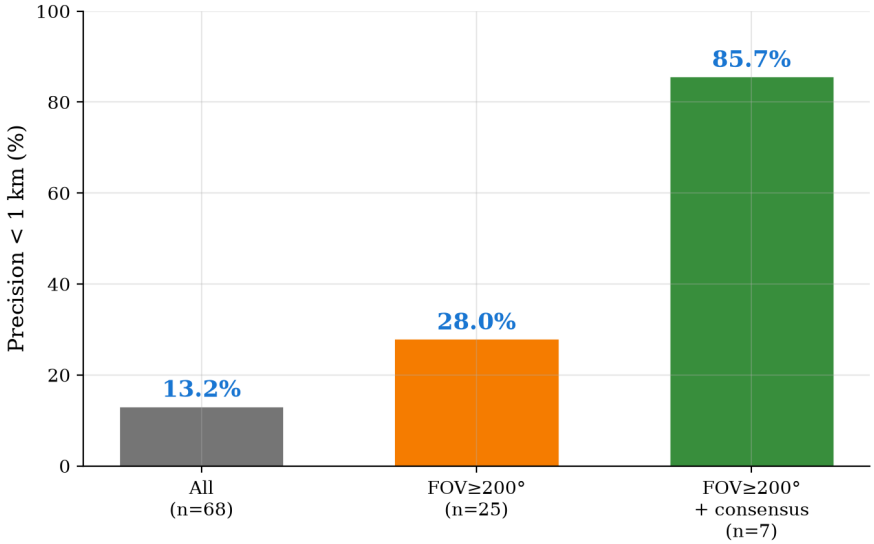

Figure 6-6. Accuracy improves with wider FOV and scorer consensus.

6.2.4. Dashboard Demo

A Streamlit-based dashboard demonstrates the full pipeline end to end. Figure 6-7 shows multiple Google Street View crop images processed through the system: each crop is individually segmented, their skyline profiles are fused into a single wide-FOV profile, and the result is matched against the database. The dashboard overlays the extracted profile against the best-matching database profile and marks the estimated location on a map with the distance and direction to the nearest known landmark.

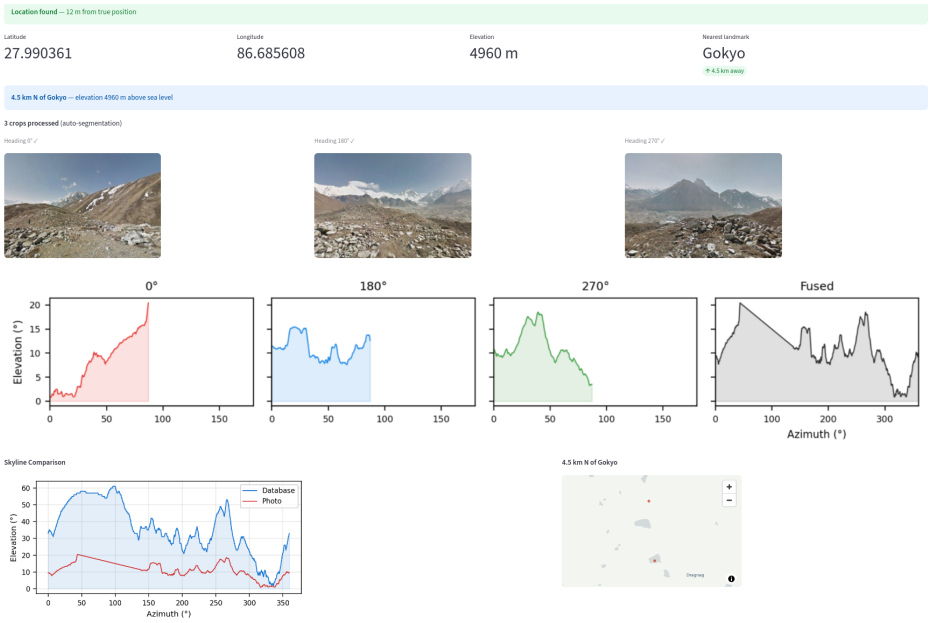

Figure 6-7. Streamlit dashboard: multi-crop photos segmented and fused, skyline comparison, and location map.

6.2.5. Failure Analysis

Two patterns emerge among rejected panoramas. The first is imposter mimicry, where nearby DEM grid points share similar skyline profiles due to valley symmetry at 30 m resolution, causing the matcher to select the wrong location. The second is narrow field of view, where a single crop covers too little of the horizon to distinguish one viewpoint from another. Both effects are expected at 30 m DEM resolution and are compounded by the low image quality of Google Street View in this region.

6.3. Noise Robustness

To test how well the matcher handles imperfect profiles, reference profiles were corrupted with Gaussian noise at varying standard deviations (σ) and matched back against the database. Five reference profiles are tested to verify that the matcher tolerates reasonable profile noise.

Table 6-5. Noise robustness

Noise σ	**<100 m**	**<1 km**
0.0°	3/5	3/5
0.25°	3/5	3/5
0.5°	3/5	3/5
1.0°	3/5	3/5
2.0°	2/5	3/5
`uint8` quantization	3/5	3/5

Accuracy is stable up to 1.0° noise. The two failures at 0.0° are valley-symmetry imposter matches — they fail regardless of noise because the terrain is ambiguous at 30 m DEM resolution.

6.4. Summary

- Synthetic: 86.4% of answered queries within 500 m, median 135 m.
- Real-photo: with consensus filtering, 85.7% within 1 km, median ~40 m.
- Upper bound: correct location found every time with clean profiles.
- Noise robustness: stable up to 1.0°.

7. CONCLUSION

7.1. Summary

This project built a camera-based location estimator for mountain terrain. The system has three main components: a U-Net segmentation network that separates sky from terrain in photographs, a precomputed database of 1.34 million horizon profiles generated from a 30 m DEM, and a three-stage matching pipeline that uses multiple scorers fused by Reciprocal-Rank Fusion. A small mobile prototype verified that the entire pipeline runs on Android hardware without network connectivity.

7.2. Key Results

- **Synthetic:** 86.4% of answered queries within 500 m, median 135 m.
- **Real-photo:** with consensus filtering, 85.7% within 1 km, median ∼40 m.
- **Robustness:** stable up to 1.0° profile noise.
- **On-device:** 41 MB model, 10.5 MB database subset, ∼138 MB peak RAM.

7.3. Limitations

- **DEM resolution:** At 30 m, nearby viewpoints share similar profiles. Finer DEMs would reduce imposter matches.
- **Segmentation quality:** Real photos with haze, fog, or unusual lighting produce noisier masks. The consensus gate handles this by abstaining.
- **Multiple photos requirement:** A single cropped photo captures only 60°–80° of the horizon. Several viewpoints produce similar partial profiles, leading to ambiguous matches. The system compensates by allowing two or three overlapping photos at different headings, which are stitched into a wider profile before matching.
- **Study area scope:** Evaluation was limited to the Khumbu region. Performance in other mountain ranges with different terrain geometry has not been tested.

7.4. Future Work

- **Finer DEMs:** Higher-resolution elevation data would reduce profile ambiguity and improve matching accuracy.
- **Field testing:** Real hiking conditions with phone cameras and varying weather would validate the system under operational constraints.
- **Other regions:** Testing on different mountain ranges would check whether the approach generalises beyond the Khumbu valley.
- **Improved segmentation:** Training on larger, more diverse mountain datasets could reduce the segmentation error that currently limits real-photo accuracy.

A. MATHEMATICAL DERIVATIONS

Derivations for the physical models and signal-processing justifications used in the main text.

A.1. Curvature Drop

Over a spherical Earth of radius R_E, an observer at height h_o sees a horizon at distance d. From the right triangle formed by the Earth centre, the observer, and the tangent point:

$$(R_E + h_o)^2 = R_E^2 + d_{\text{horizon}}^2. \tag{A-1}$$

Expanding and neglecting $h_o^2 \ll R_E h_o$:

$$d_{\text{horizon}} \approx \sqrt{2R_E h_o}. \tag{A-2}$$

A terrain feature at distance d appears lower because the reference horizontal curves downward by:

$$\delta(d) = \frac{d^2}{2R_E}. \tag{A-3}$$

Standard atmospheric refraction effectively increases visible range. Replacing R_E with $R_E/(1-k)$ where $k \approx 0.13$:

$$h(d) = \frac{d^2(1-k)}{2R_E}, \qquad k = 0.13. \tag{A-4}$$

Worked example.

With $R_E = 6371\,\text{km}$, $k = 0.13$:

Distance d (km)	Hidden height h (m)
10	49
20	196
30	441
40	785
50	1,226

At 30 km, any ridge below $\sim$ 440 m above the observer's eye level is hidden. This justifies the 30 km ray-trace limit.

A.2. Koschmieder Visibility Law

The Koschmieder law relates apparent contrast $C(d)$ of an object at distance d to its inherent contrast C_0:

$$C(d) = C_0 \, \exp(-\sigma_{\text{ext}} \, d), \tag{A-5}$$

where σ_{ext} is the volume extinction coefficient. The meteorological visibility V is the distance at which contrast drops to 2%:

$$V = \frac{\ln(1/0.02)}{\sigma_{\text{ext}}} = \frac{3.912}{\sigma_{\text{ext}}}. \tag{A-6}$$

Substituting:

$$C(d) = C_0 \, \exp\!\left(-\frac{3.912 \, d}{V}\right). \tag{A-7}$$

Contrast table for $V = 50$ km (clear mountain air).

Distance d (km)	Relative contrast C/C_0 (%)
5	68.2
10	46.5
15	31.7
20	21.6
25	14.7
30	10.0
40	4.7
50	2.2

At 30 km, contrast has fallen to 10%. This reinforces the 30 km limit: terrain features beyond this range contribute unreliable skyline information.

A.3. Cross-Correlation via FFT

The Pearson normalised cross-correlation between two zero-mean signals f and g of length N at lag τ is:

$$r(\tau) = \frac{\sum_{i=0}^{N-1} f(i) \, g((i+\tau) \bmod N)}{\sqrt{\sum_i f(i)^2 \cdot \sum_i g(i)^2}}. \tag{A-8}$$

The numerator is a circular cross-correlation. By the correlation theorem:

$$(f \star g)[\tau] = \text{IFFT}\big(\text{FFT}(f) \odot \overline{\text{FFT}(g)}\big)[\tau], \tag{A-9}$$

where $\star$ denotes circular cross-correlation, $\odot$ is element-wise multiplication, and $\overline{(\cdot)}$ is complex conjugation. This reduces computation from $O(N^2)$ to $O(N \log N)$.

B. COST ESTIMATE

The project was developed entirely using open-source software and publicly available data. No hardware was purchased. Table B-1 itemises the costs incurred.

Table B-1. Project cost breakdown.

Item	Cost (NPR[1])	Remarks
Copernicus GLO-30 DEM data	0	Open access
PyTorch, ONNX Runtime, Horayzon, PyRender	0	Open source
Python, Git, VS Code	0	Open source
Kivy mobile framework	0	Open source
Google Street View imagery (evaluation)	0	Public
Personal laptop (development)	0	Existing
Google Colab (GPU training)	0	Free tier
Report printing and binding	~2000	Department requirement
Total	~2000	

C. PROJECT TIMELINE

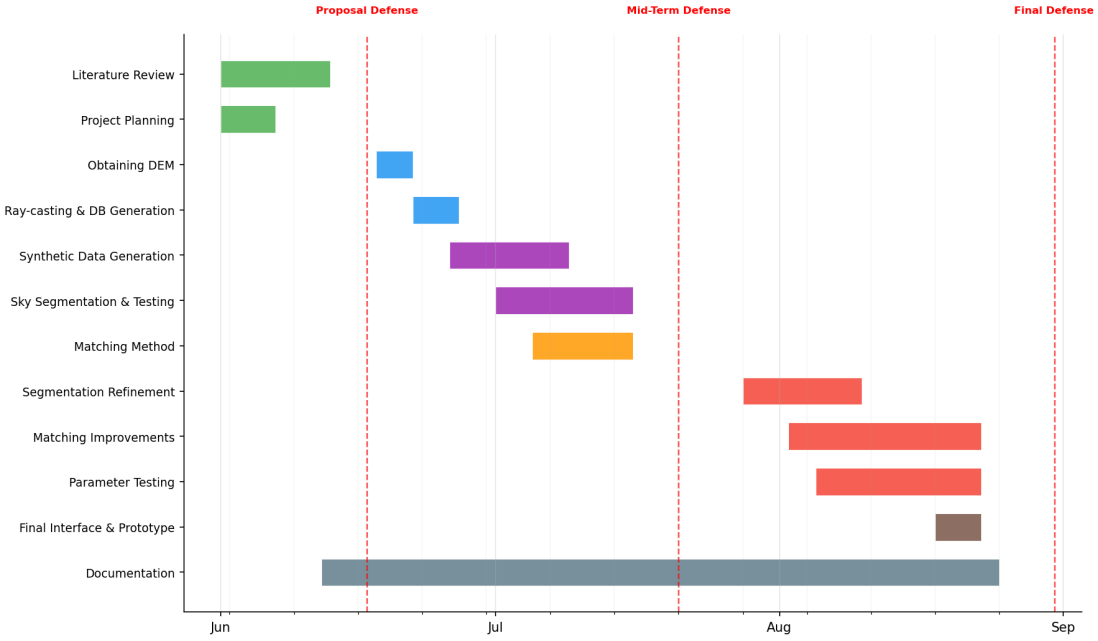

Figure C-1. Project Timeline.

D. PIPELINE DIAGRAMS

D.1. Sky Mask Post-Processing

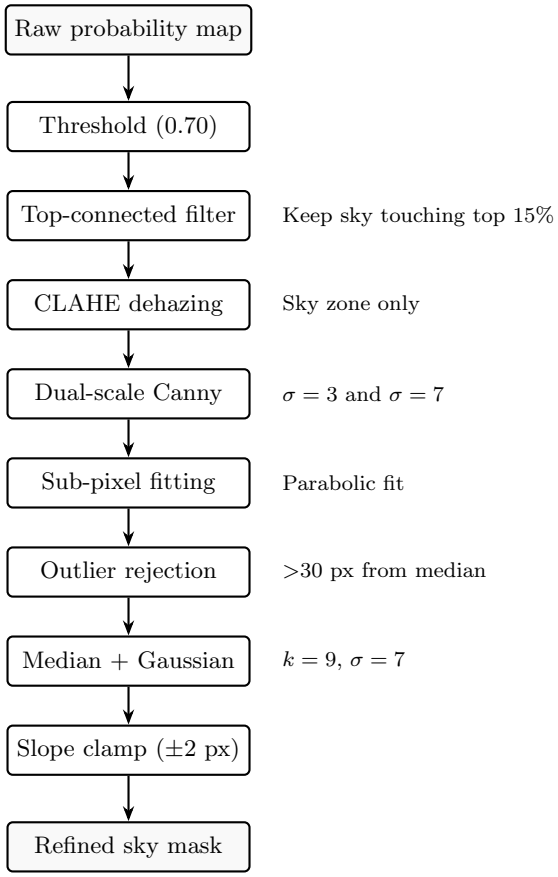

Figure D-1. Post-processing pipeline. Each step addresses a specific failure mode: thresholding gives a rough mask, top-connected filtering removes false sky blobs, CLAHE restores contrast in fog, Canny detects the true boundary, sub-pixel fitting improves precision, and smoothing removes remaining noise.

D.2. Matching Pipeline

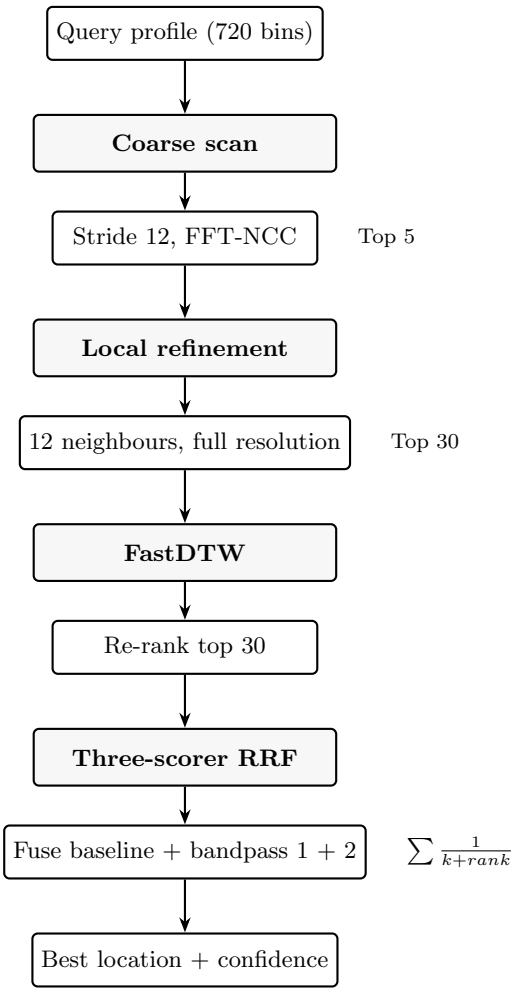

Figure D-2. Matching pipeline. The query profile passes through three stages of progressively finer search, then three scorers at different terrain scales are fused using Reciprocal-Rank Fusion.

D.3. System Activity Flow

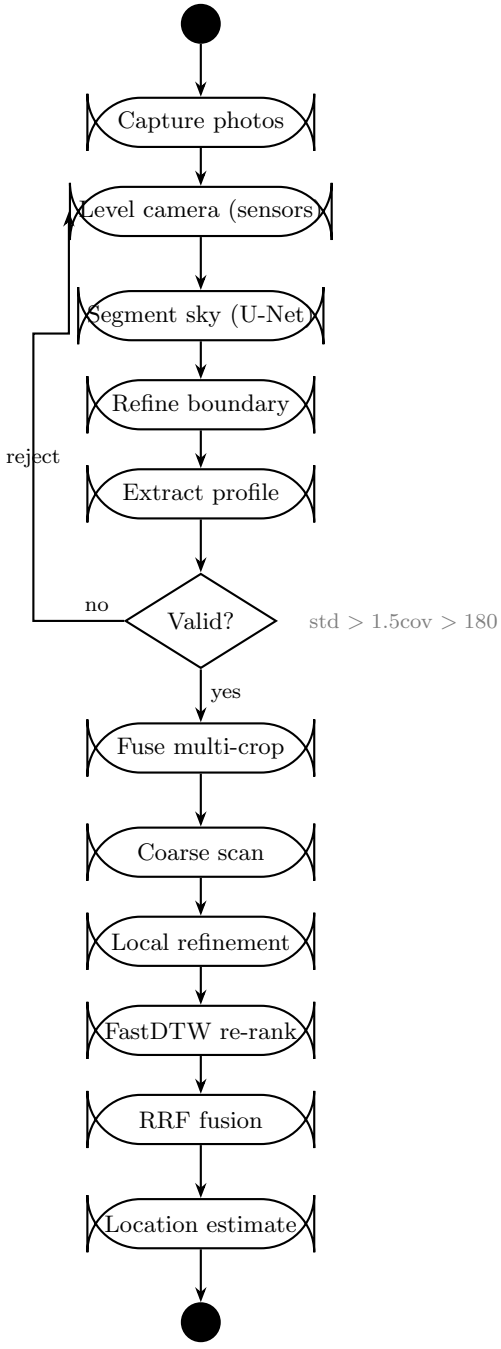

Figure D-3. System activity flow from user capture to location estimate. The feedback loop on the left rejects profiles that fail the quality gate and prompts the user to retake photos.

E. DATASET SAMPLES

Figure E-1 shows sample images from each dataset used in this project.

- **GeoPose3K** (row 1): Mountain photographs with annotated sky-terrain boundaries, taken in the European Alps. Used for training the segmentation network.
- **Google Street View** (row 2): Panoramic crops from the Khumbu region. Note the fog, haze, and low resolution in most images, which motivates the filtering described in Chapter 6.
- **Sky photos** (row 3): Cloud and sky photographs used as backdrops in synthetic training data. Combined with terrain meshes under different weather presets.
- **Satellite texture** (row 4): Esri World Imagery cropped to the study area. Textured onto the DEM mesh for realistic terrain rendering.

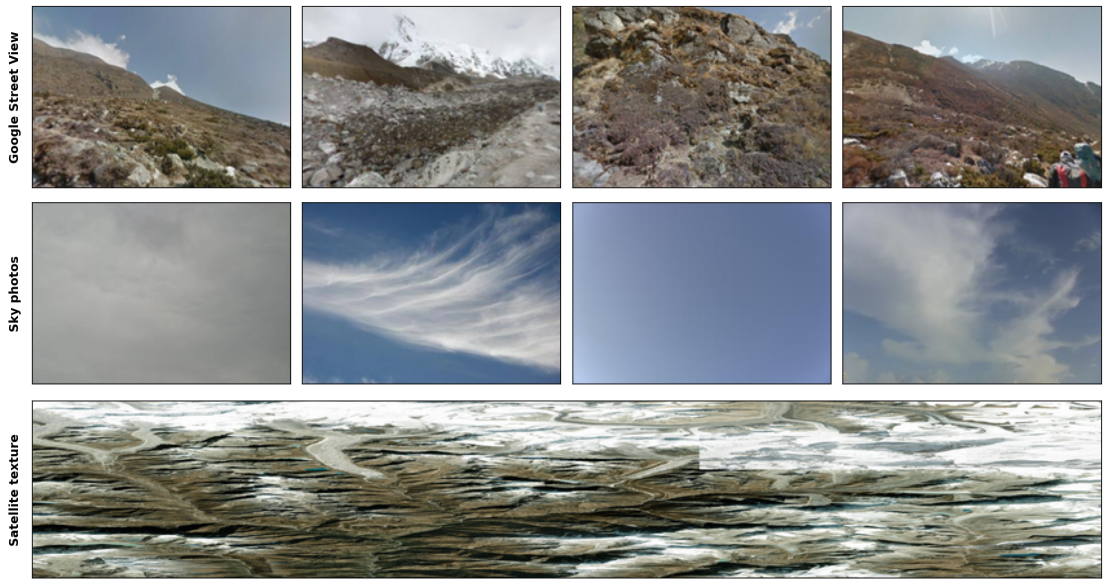

Figure E-1. Sample images from each dataset: GeoPose3K, Google Street View, sky photos, satellite texture, and synthetic output.

REFERENCES

[1] P. C. Naval, M. Mukunoki, M. Minoh, and K. Ikeda, "Estimating camera position and orientation from geographical map and mountain image," in *38th Pattern Sensing Group Research Meeting, Society of Instrument and Control Engineers*, 1997, pp. 9–16.

[2] H. Gupta and S. Brennan, "Camera pose estimation by alignment of mountain peaks," *Computer Vision and Image Understanding*, vol. 110, no. 3, pp. 311–324, 2008.

[3] G. Baatz, O. Saurer, K. Köser, and M. Pollefeys, "Large scale visual geo-localization of images in mountainous terrain," in *European Conference on Computer Vision (ECCV)*, 2012, pp. 517–530.

[4] O. Saurer, G. Baatz, K. Köser, L. Ladický, and M. Pollefeys, "Image based geo-localization in the Alps," *International Journal of Computer Vision*, vol. 116, no. 3, pp. 213–225, 2016.

[5] H. Li, J. Zhao, B. Yan, L. Yue, and L. Wang, "Global DEMs vary from one to another: An evaluation of newly released Copernicus, NASA and AW3D30 DEM on selected terrains of China using ICESat-2 altimetry data," *International Journal of Digital Earth*, vol. 15, no. 1, pp. 1149–1168, 2022.

[6] Y. Chen, G. Qian, K. Gunda, H. Gupta, and K. Shafique, "Camera geolocation from mountain images," in *18th International Conference on Information Fusion (Fusion)*. IEEE, 2015, pp. 1587–1596.

[7] Z. Pan, J. Tang, T. Tjahjadi, X. Xiao, and Z. Wu, "Camera geolocation using digital elevation models in hilly area," *Applied Sciences*, vol. 10, no. 19, p. 6661, 2020.

[8] Z. Pan, J. Tang, T. Tjahjadi, and F. Guo, "Fast geo-location method based on panoramic skyline in hilly area," *ISPRS International Journal of Geo-Information*, vol. 10, no. 8, p. 537, 2021.

[9] J. Brejcha and M. Čadík, "GeoPose3K: Mountain landscape dataset for camera pose estimation in outdoor environments," Faculty of Information Technology, Brno University of Technology, Tech. Rep., 2019.

[10] S. W. Yang, I. C. Kim, and J. S. Kim, "Robust skyline extraction algorithm for mountainous images," in *Proceedings of the Second International Conference on Computer Vision Theory and Applications (VISAPP)*, 2007, pp. 253–257.

[11] T. Ahmad, E. Emami, M. Čadík, and G. Bebis, "Resource efficient mountainous skyline extraction using shallow learning," in *International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2021, pp. 1–9.

[12] J. Tomešek, M. Čadík, and J. Brejcha, "CrossLocate: Cross-modal large-scale visual geo-localization in natural environments using rendered modalities," in *IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2022, pp. 3174–3183.

[13] Z. Liu, F. Guo, H. Liu, X. Xiao, and J. Tang, "CMLocate: A cross-modal automatic visual geo-localization framework for a natural environment without GNSS information," *IET Image Processing*, vol. 17, no. 12, pp. 3524–3540, 2023.

[14] S. Salvador and P. Chan, "Fastdtw: Toward accurate dynamic time warping in linear time and space," in *Proceedings of the 3rd Workshop on Mining Temporal and Sequential Data*. ACM, 2007.